

AMCT-LLM量化能力发布解读： 主流模型与PTQ算法实践

作者：章奕

时间：2026.8.19

目录

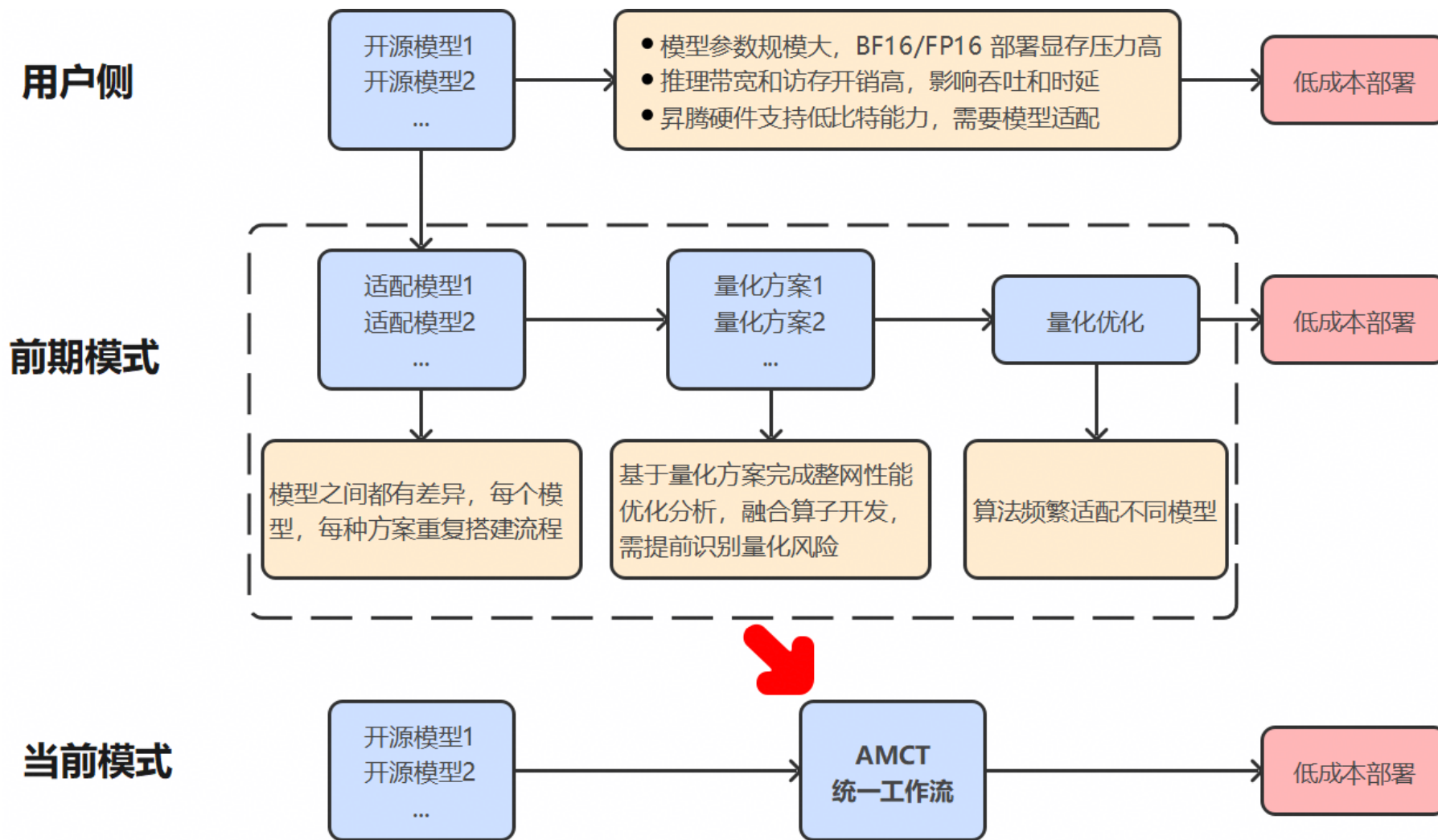
Part 1 AMCT-LLM整体设计

Part 2 核心模块拆解

Part 3 一站式样例演示

Part 4 Agent优化 workflow

工具设计背景



简化整体流程，避免复杂繁琐重复的工作，搭建一体式工作流

AMCT-LLM 定位

AMCT

<https://gitcode.com/cann/amct>



CANN-RECIPES-INFER

<https://gitcode.com/cann/cann-recipes-infer>

作为CANN生态一环，与CANN-RECIPES-INFER强关联，AMCT工具量化优化的模型可一键部署

CANN-RECIPES-INFER

基于 CANN 平台的大模型推理优化实践

覆盖 DeepSeek、Qwen、GLM、Hunyuan 等主流模型，克隆即用，快速上手昇腾 NPU 推理，复现生产级高性能

License Apache 2.0 Models 23+

[概述](#) · [快速开始](#) · [样例列表](#) · [社区讨论](#)

- | | | | |
|---|-------------------------------------|--------------------------------------|---------------------------------|
| > deepseek_r1 | > glm_5_2 | > kimi_k2_thinking | > qwen3_5 |
| > deepseek_v3_2_exp | > gpt_oss | > longcat-2.0 | > qwen3_moe |
| > deepseek_v4 | > hstu | > longcat_flash | > sana-video |
| > deepseek_v4_flash_tilelang_ | > hunyuan-image-3.0 | > longcat_flash_lite | > step3p7_flash |
| > gemma_4 | > hunyuan-video | > pangu_7b | > wan2.2-i2v |
| > glm_5 | > hy3 | > qwen | |

前期：infer仓中大部分模型都要**量化人员定点优化**，同时将量化系数传到OBS桶，每个模型提供**定制化转换脚本**

当前：新模型量化，AMCT**提供通用接口**，权重可一键对接infer仓，同时适配sglang和vllm等框架

AMCT-LLM 能力总结

20+

模型支持

7

低bit浮点格式

10+

算法

01 覆盖主流模型

支持 DeepSeek v3.2、DeepSeek v4、Qwen3、Qwen3-Next、Qwen3.5/3.6、GLM 5.1/5.2、Longcat Flash/Next、HunYuan3.0等主流模型，覆盖Dense/MoE, Attention(GQA, MHA, MLA)等典型结构

02 量化格式

支持Atlas-A3 Pod/950PR/DT上的INT4/8、FP8、MXFP4/8、HiF8六种格式以及下一代的HiF4格式，快速验证同一模型在不同格式下的精度损失

03 多种量化算法

支持同一模型在不同算法上的精度表现，快速沉淀面向相同架构下的精度补偿recipe，目前已支持LAC、LWC、AutoRound、SVDQuant、GPTQ、Hadamard(可学习)、Flatquant、OmniQuant、SmoothQuant等针对权重 outlier、激活范围、舍入误差、块内分布等问题选择优化算法

Adapt
模型适配



Evaluate
精度评估



Quantize
PTQ 优化



Deploy
部署导出

AMCT 将模型适配、精度评估、量化优化、部署导出收敛为统一流程

AMCT-LLM 核心模块-模式适配

Adapt
模型适配

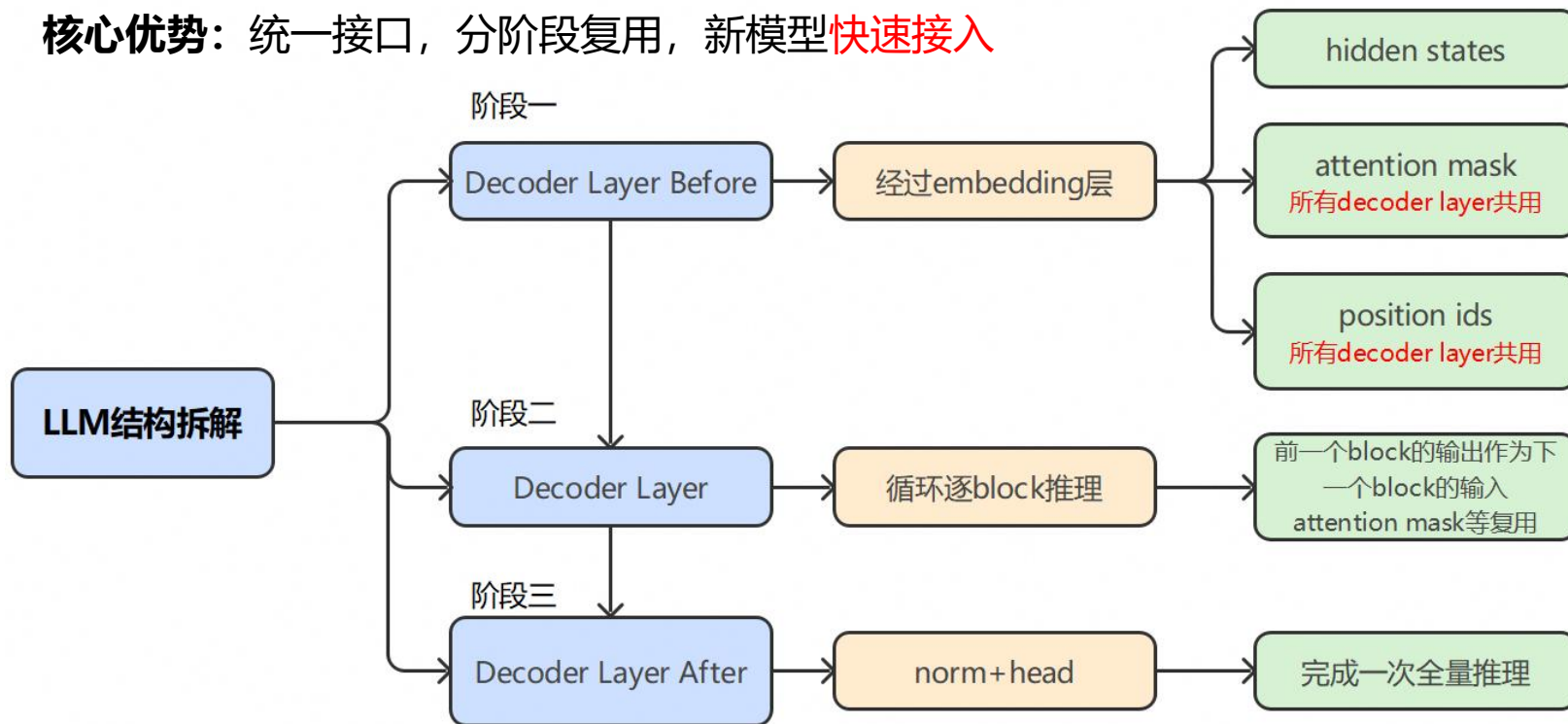
Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

LLM共性： 现有LLM均为decoder-only结构，因此可以拆成以下三个阶段完成整个模型推理来评测WikiText ppl

核心优势： 统一接口，分阶段复用，新模型**快速接入**



基础用法： 以下为不量化下的浮点推理评测脚本示例，新模型导入之后通过传参：

```
--model 指定权重路径, --model_name 指定注册的模型名
python -m amct_pytorch.eval \
    --model ./Qwen3-235B \
    --model_name qwen3_moe --seq_len 4096 --granularity block \
    --device npu:0 --eval_mode bf16
```

代码接口映射

```
# 以注册机制, 集成基类, 适配qwen系列模型
@MODEL_REGISTRY.register(
    name="qwen3",
    task="llm",
    family="qwen",
    description="Qwen3 dense model adapter",
)
```

```
# 部分模型embed和head权重共享
def load_embed_state_dict()
# 通过hook抓取embed的输出, 以及decoder的所有输入
def do_embedding_forward
```

```
# block构建较复杂, 下一页展开
def block()
# block循环调用, 中间结果online or offline
def do_block_forward()
```

```
# head的输出, 统一调eval接口
def do_head_forward()
```


AMCT-LLM 核心模块-模式适配

Adapt
模型适配

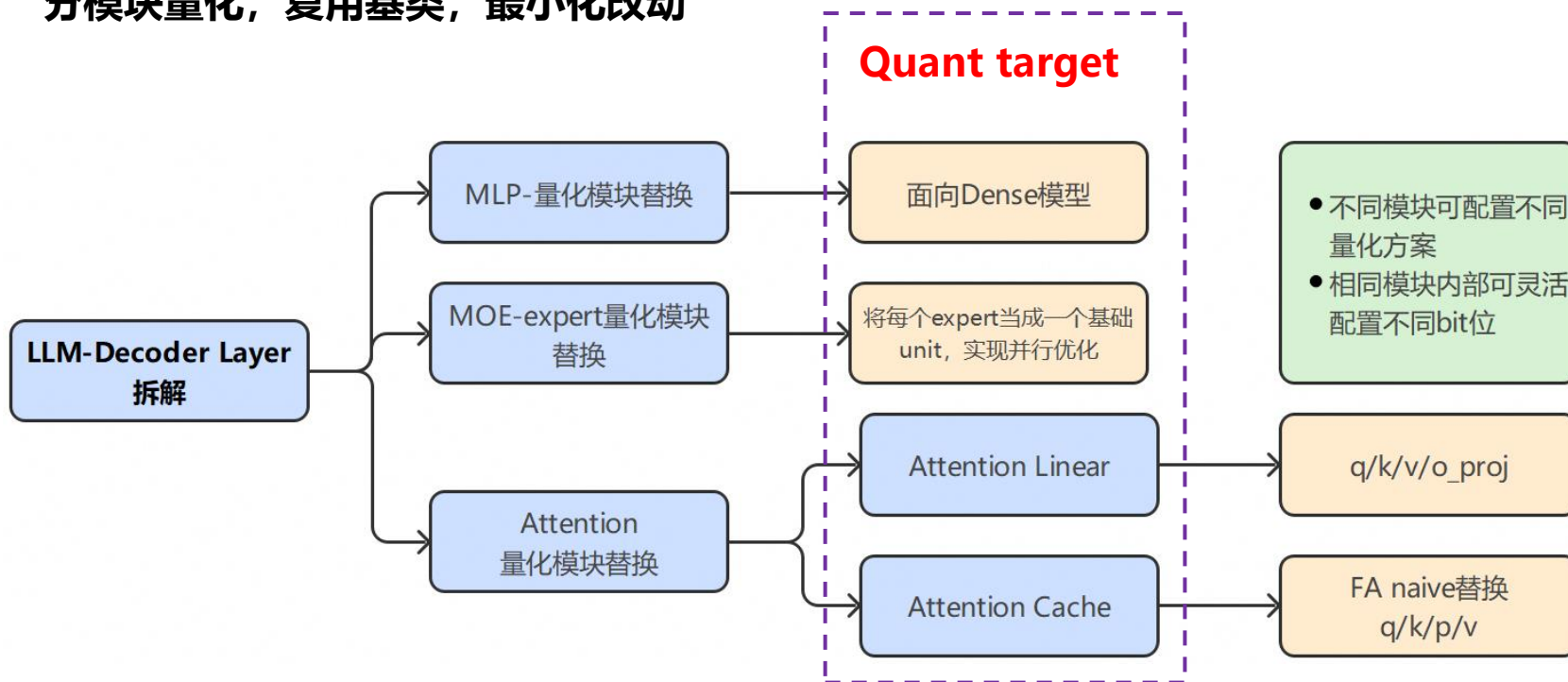
Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

通用模块拆解：模型量化对象均集中在Decoder Layer中，将其按以下方式拆解，针对其中量化模块灵活配置量化方案

分模块量化，复用基类，最小化改动



基础用法：根据传参：

`--quant_target choices=["mlp", "moe", "attn-linear", "attn-cache"]` 来指定量化的模块

代码接口映射

```
# 通过统一接口构建quant block
def build_quant_block()
```

```
# 继承基类，稀疏/稠密最终都指向MLP结构
class QuantQwen3MLP(QuantGatedMLP):
    pass
```

```
# Attention继承源码，构建量化节点，FA计算
需要统一替换方便cache量化
class QuantQwen3Attn(Qwen3Attention):
```

```
# head的输出，统一调eval接口
def do_head_forward()
```

AMCT-LLM 核心模块-模式适配

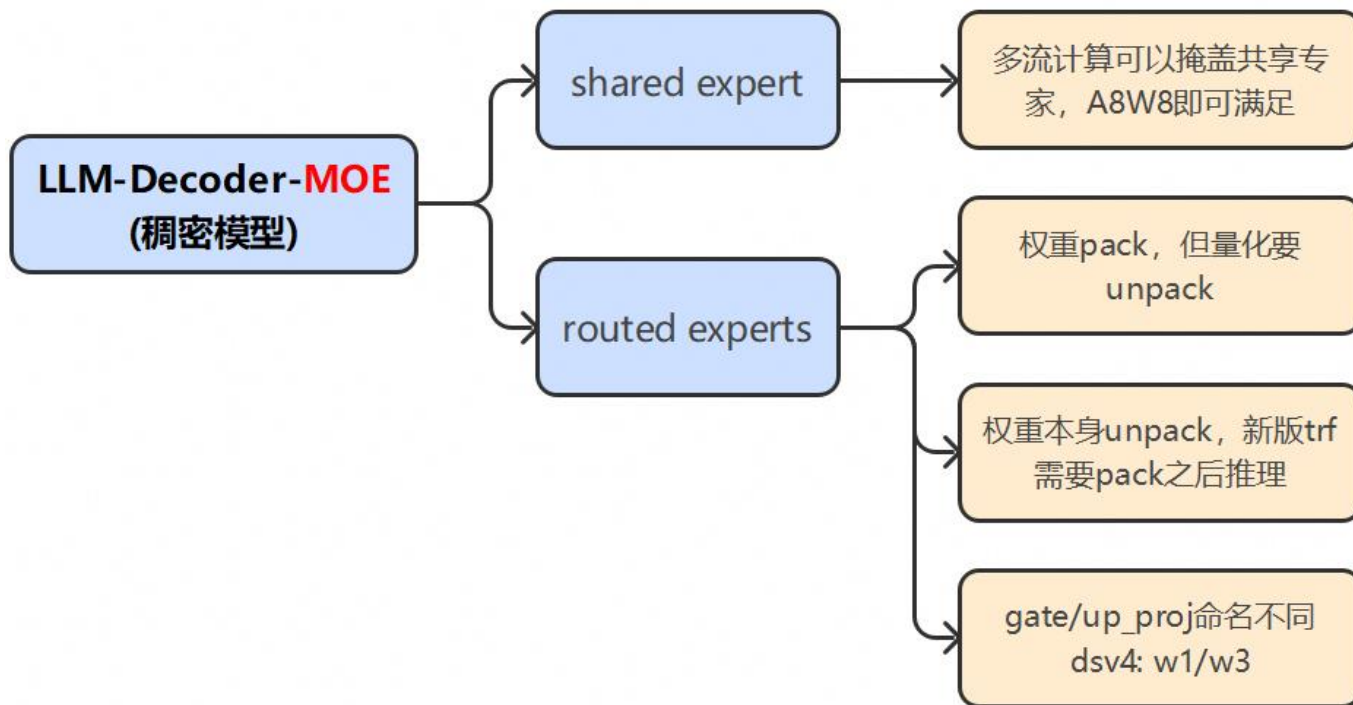
Adapt
模型适配

Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

Decoder Layer MOE替换示例:



pack/unpack: MoE 推理瓶颈不是单个expert很大, 而是expert很多、token分发稀疏、每个expert处理的token不均匀, naive的实现是python循环逐个处理expert, **pack**即将各个**独立**的**expert权重** stack 成一个 **3D tensor**实现加速, 但量化PTQ中需要**unpack**为**每个expert单独**优化

代码接口映射

```
# 通过统一接口构建quant block
def pack_gated_expert_weights()
    开源权重:
    experts.0.up_proj.weight
    experts.1.up_proj.weight
    ...
    experts.N.up_proj.weight
    ↓ pack
    experts.up_proj.weight (新版transformers+逐
    block推理模式需要重新实现)
```

```
# 将3D tensor拆解成单个expert的单个linear,
class ExpertLinearView()
```

```
# 封装通用MLP, 最大化复用
class QuantV4Expert(QuantGatedMLP)
```


AMCT-LLM 核心模块-精度评估

Adapt
模型适配

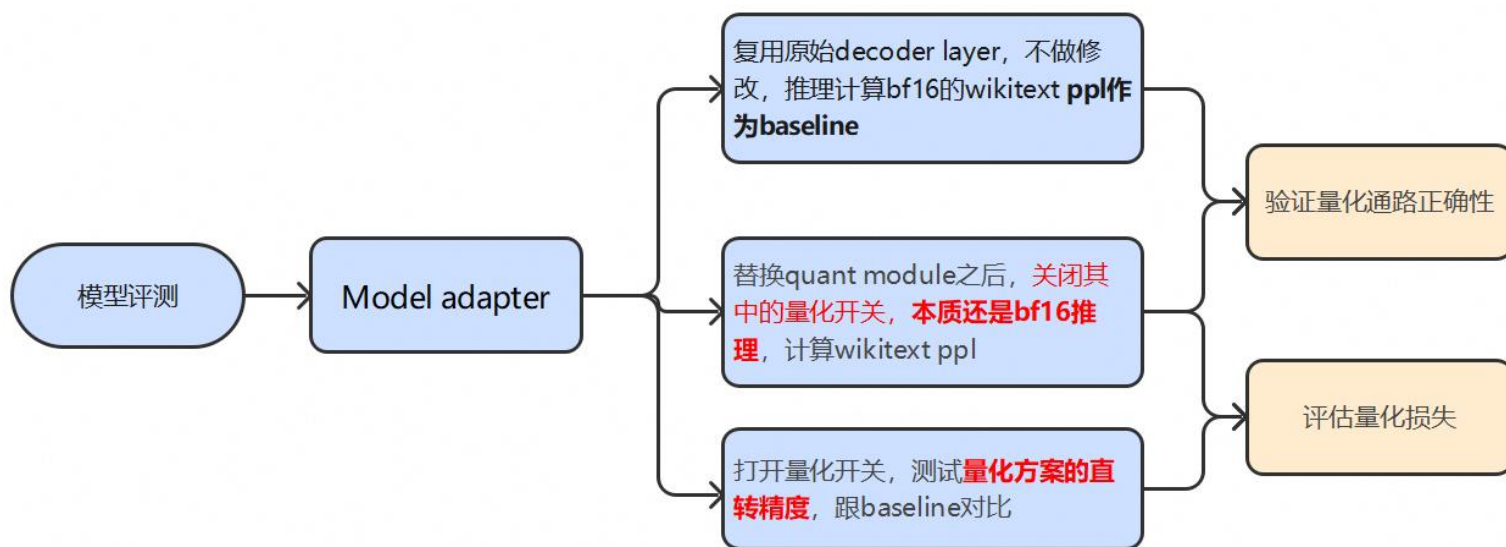
Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

WikiText: 一个标准语言建模数据集，来源于 Wikipedia 中高质量文章，常用于用 PPL 衡量语言模型建模能力。通过PPL可以快速衡量模型量化前后语言建模能力有没有明显退化，其轻量、稳定的特点，适合在PTQ调参过程中快速评估算法效果。

三通路并行执行，验证正确性后再执行量化评估



基础用法

baseline: 以bf16评测:

```
python -m amct_pytorch.eval \
  --model ./Qwen3-235B \
  --model_name qwen3_moe --seq_len 4096 \
  --granularity block --device npu:0 --eval_mode bf16
```

通路验证: 替换需要量化的module, 关闭量化开关:

```
python -m amct_pytorch.eval \
  --model ./Qwen3-235B \
  --model_name qwen3_moe --seq_len 4096 \
  --granularity block --device npu:0 --eval_mode quant \
  --quant_target moe \
  --quant_dtype mxfp \
  --bit_config amct_pytorch/configs/bf16.yaml
```

量化评估: 打开量化开关, 对moe模块做量化直转评估精度损失, 评估是否需要进行下一步PTQ优化:

```
python -m amct_pytorch.eval \
  --model ./Qwen3-235B \
  --model_name qwen3_moe --seq_len 4096 \
  --granularity block --device npu:0 --eval_mode quant \
  --quant_target moe \
  --quant_dtype mxfp \
  --bit_config amct_pytorch/configs/w4a4.yaml
```

AMCT-LLM 核心模块-格式适配

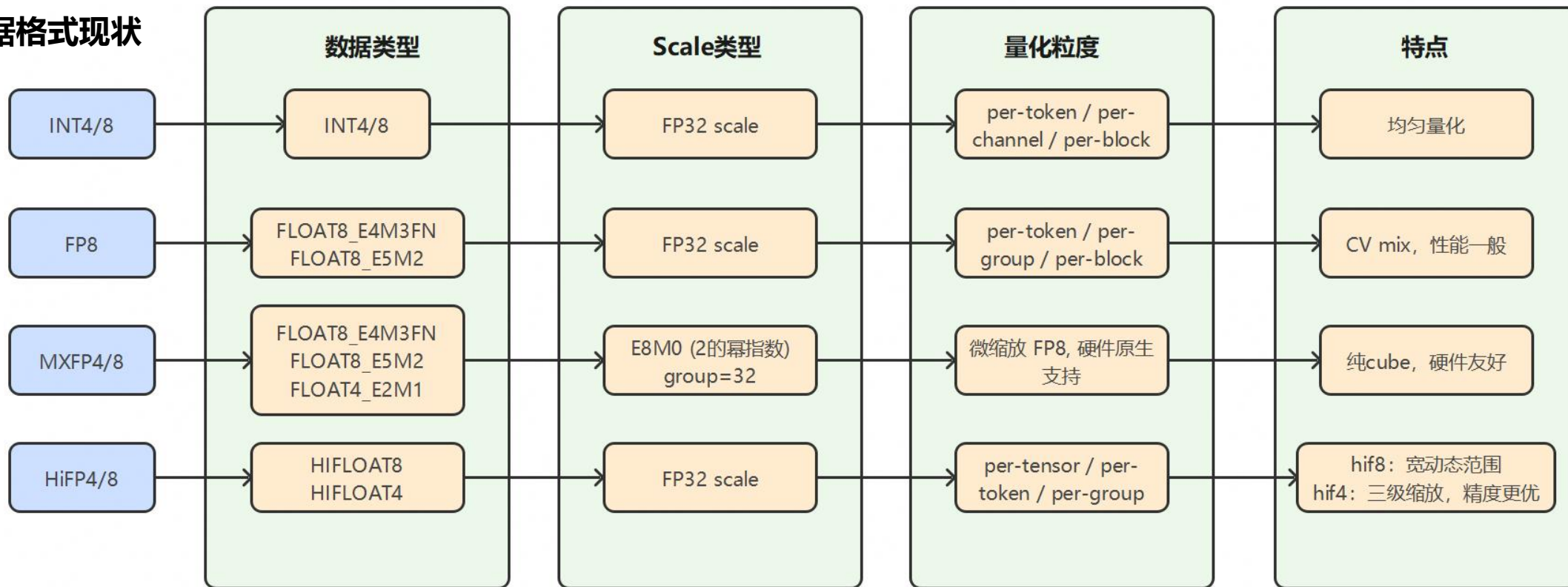
Adapt
模型适配

Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

数据格式现状



用法: 通过传参 `--quant_dtype` 指定数据类型 `choices=['int', 'mxfp', 'hifp']`, `--bit_config` 指定量化的yaml文件路径, 里面标明量化各层bit

```
# 指定全模块bit位  
w_bits: 8  
a_bits: 8
```

```
# moe某一层做4bit  
routed:  
  down_proj:  
    { w_bits: 4, a_bits: 4 }
```

```
# attn-linear某一层做4bit  
o_proj:  
  { w_bits: 4, a_bits: 4 }
```

```
# attn-cache某一层做4bit  
k: 4
```

AMCT-LLM 核心模块-格式适配

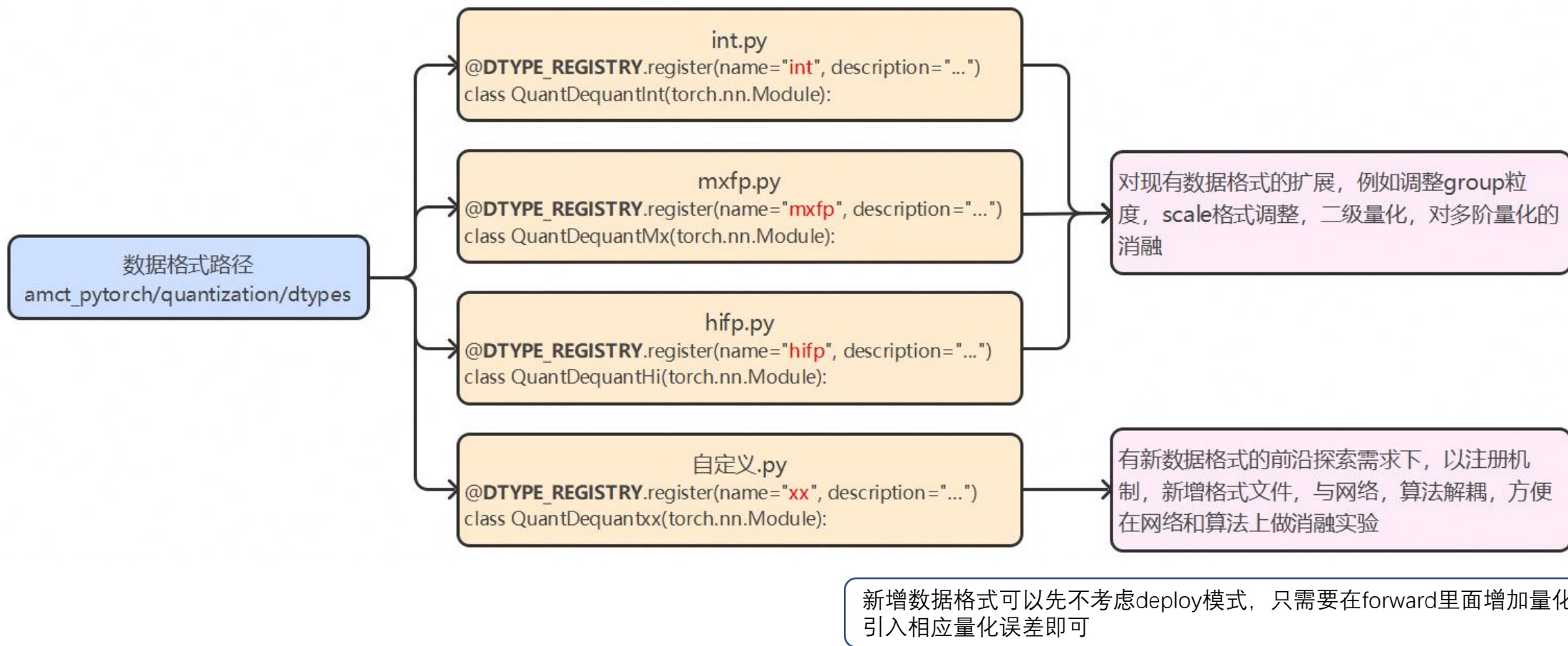
Adapt
模型适配

Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

新数据格式开发适配：当前数据格式已适配主流数据格式，开发者如何适配探索新的数据格式？



AMCT-LLM 核心模块-算法适配

Adapt
模型适配

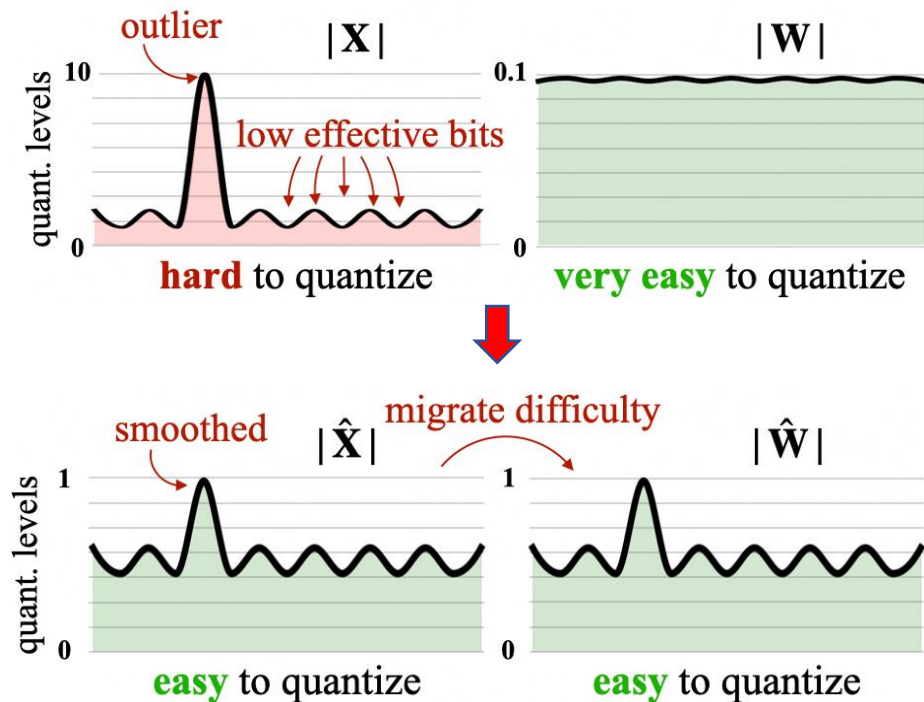
Evaluate
精度评估

Quantize
PTQ 优化

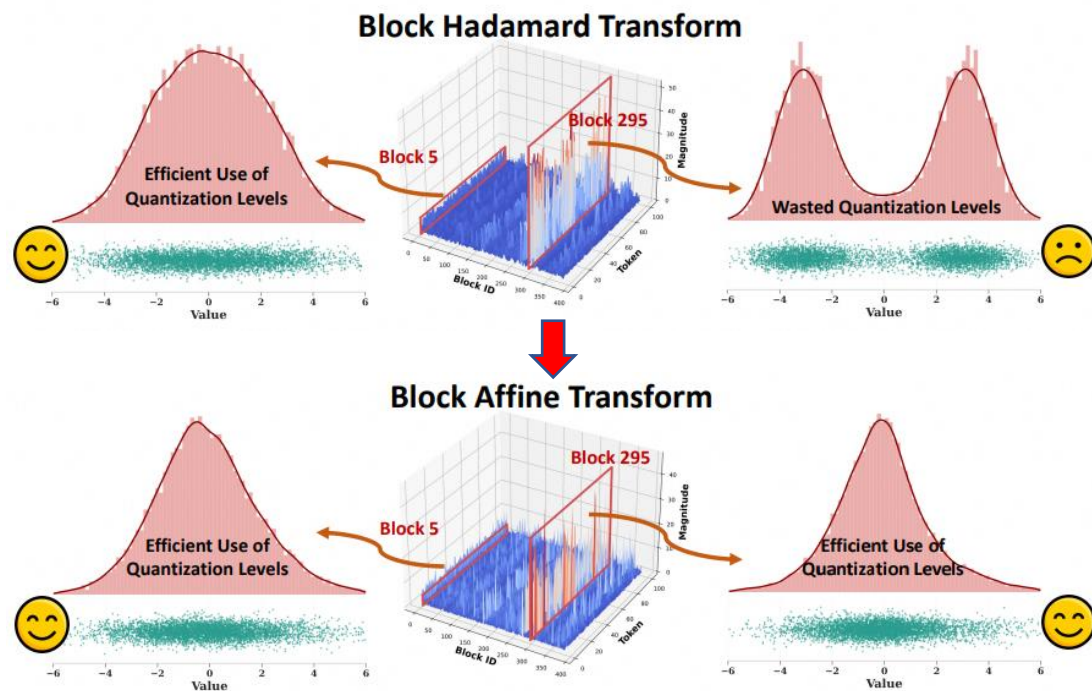
Deploy
部署导出

量化算法: outlier抑制优化量化误差

从基于统计的smoothquant -> 可以学的omniquant



从通用的正交变换 -> 仿射变换 -> per-block的仿射变换



整体趋势:

- 对A4W4的精度损失, 主要针对outlier的抑制为主要研究方向, 旋转变换逐渐成为标配(global/block + Hadamard/affine transform);
- 对A8W4的精度损失, 一般是权重引入的量化误差, 可通过outlier的截断(autoclip), 舍入误差来优化(autoround)
- 组合算法: 整网的量化方案, 一般会叠加多种量化算法来优化精度损失

AMCT-LLM 核心模块-算法适配

Adapt
模型适配

Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

工具内量化算法选型与分类

面向激活 (仅作用于激活)

LAC (Learnable activation clipping)
仅作用激活的量化算法较少, 一般A8
精度有损下用LAC基本够用, A4下会面
向结构变换做outlier的转移

面向权重 (仅作用于权重)

LWC (Learnable activation clipping)
Autoround(Learnable rounding
offsets)
SVDQuant
仅作用权重的量化算法, 一般面向W4
的精度损失

结构变换 (同时作用于权重和激活)

Omniquant
Hadamard
FlatQuant
同时作用于权重和激活, 一般面向
A4W4的精度损失

选型标准: 代码仓中的量化算法不以数量为目标, 只以质量为目标, 在各旗舰模型与下游
benchmark上经过实际验证, 后续新算法的合入也会以此严格标准进行

使用方式: 通过传参 `--algos` 指定算法 lac lwc flatquant ;支持list形式的传参, 会根据算
法注册中的targets进行自动将算法作用于目标对象, 支持多算法叠加来穿刺量化精度

算法调用示例: 替换需要量化的module, 关闭量化开关:

```
python -m amct_pytorch.ptq \  
--model ./Qwen3-235B \  
--model_name qwen3_moe --seq_len 4096 \  
--granularity block --device npu:0 --eval_mode quant \  
--quant_target moe \  
--quant_dtype mxfp \  
--bit_config amct_pytorch/configs/w4a4.yaml \  
-- algos lac lwc flatquant
```

AMCT-LLM 核心模块-算法适配

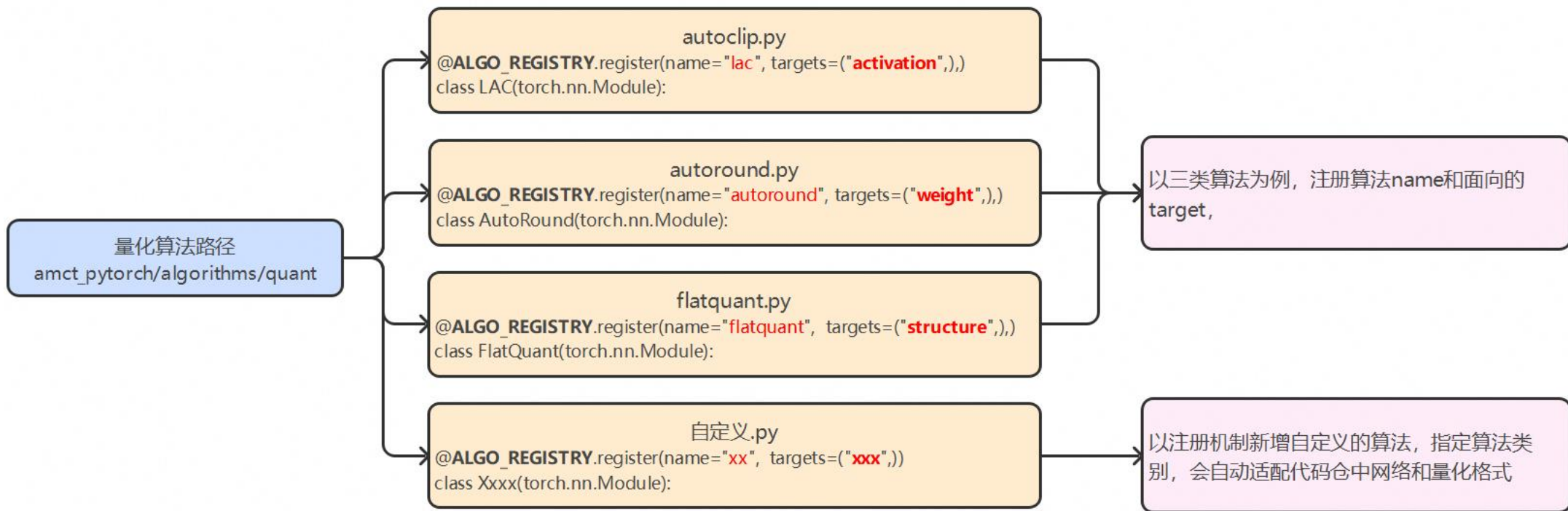
Adapt
模型适配

Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

量化算法开发适配：当前已适配常用并经过验证的算法，开发者如何适配探索新的量化算法？



算法集成模板，完善以下5个函数

```
#算法参数初始化
def __init__(self, args, ctx):
    ...
```

```
# 算法计算
def forward(self):
    ....
```

```
# 收集可学习参数，赋予梯度
def trainable_params(self):
    return list(self.parameters())
```

```
# 保存学习参数的接口，
def export_ptq_params(self):
    ...
```

```
# 加载学习参数的接口，
def load_ptq_params(self):
    ...
```


AMCT-LLM 核心模块-算法适配

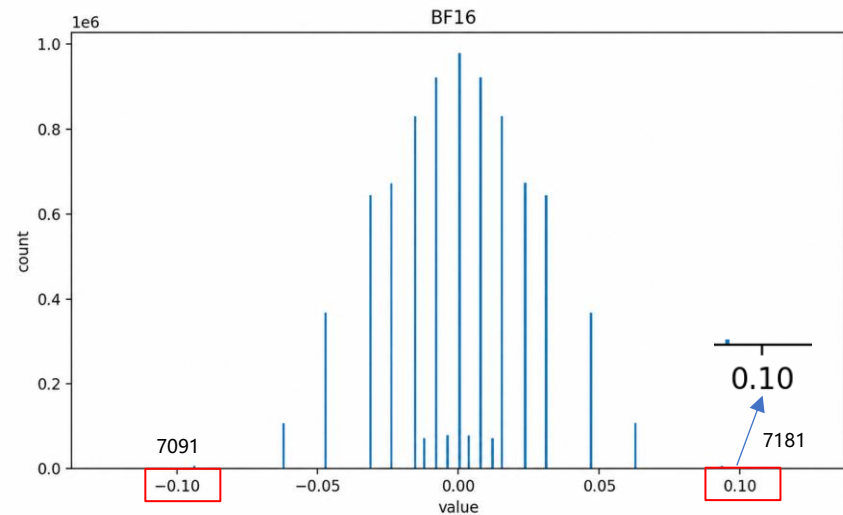
Adapt
模型适配

Evaluate
精度评估

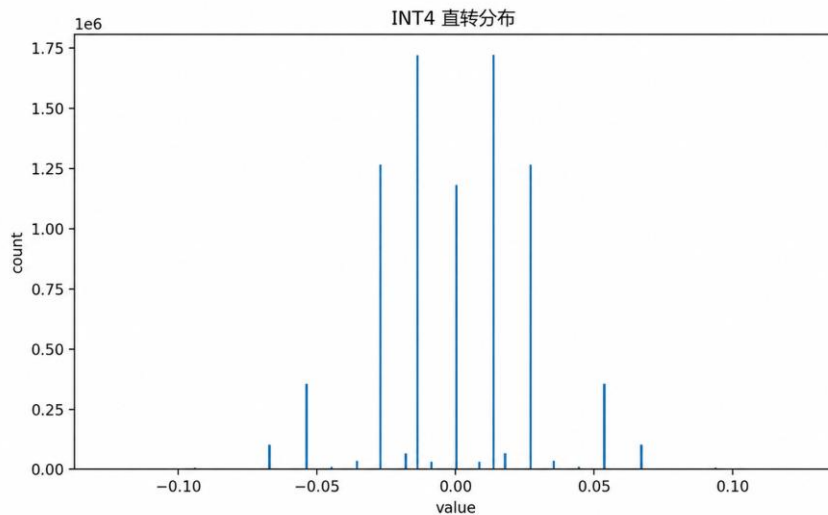
Quantize
PTQ 优化

Deploy
部署导出

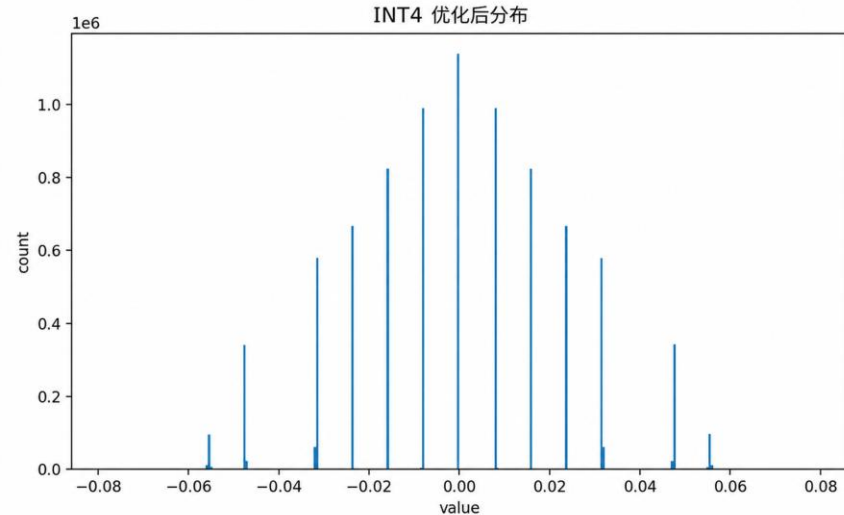
量化算法实践效果 DeepSeek-V4-Flash



原始权重bf16的直方图(*Expert 中其中一层的权重直方图，模型大部分层基本都是这种情况*)



直转INT4下，兼顾outlier值，导致主体权重分布被压缩到少量离散点上，引入较大量化误差，分布被完全打散，最终导致吐字乱码



通过LWC做outlier截断，误差最小化，整体分布重新矫正

量化方案	No Thinking							Thinking
	MMLU-Pro	CMMLU	ARC-E	ARC-C	MATH500	GPQA-Diamond	DROP-em/f1	GPQA-Diamond
Baseline	83	—	—	—	—	71.2	—	87.4
BF16	82.83	89.67	94.87	94.28	94.8	73.23	86.03	88.34
Moe-W4A8-int	82.49	89.6	94.74	94.11	94.8	73.23	87.28	87.88

AMCT-LLM 核心模块-部署导出

Adapt
模型适配

Evaluate
精度评估

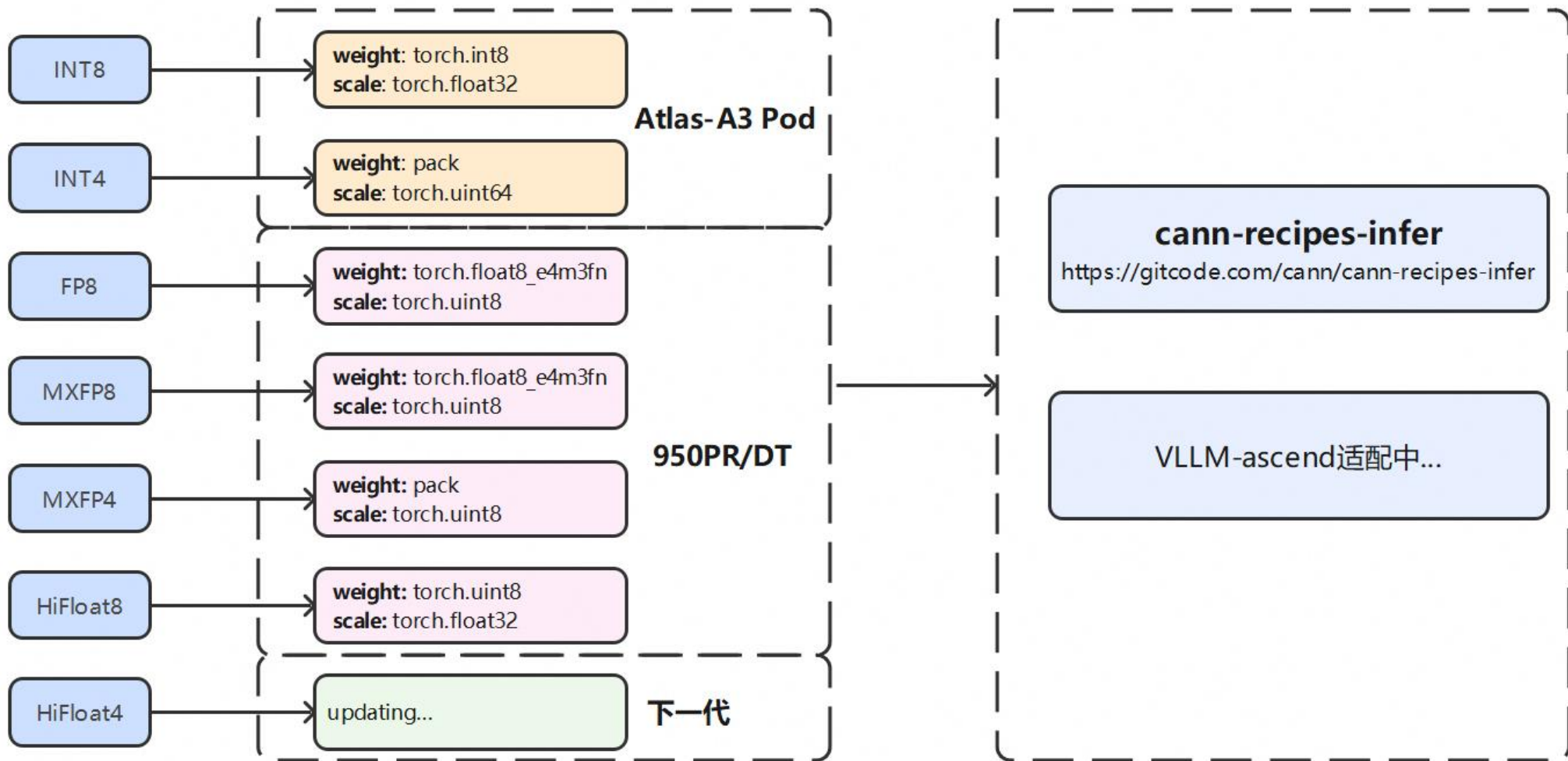
Quantize
PTQ 优化

Deploy
部署导出

部署： 量化格式

离线量化权重导出

权重部署对接下游仓



AMCT-LLM 一站式样例演示

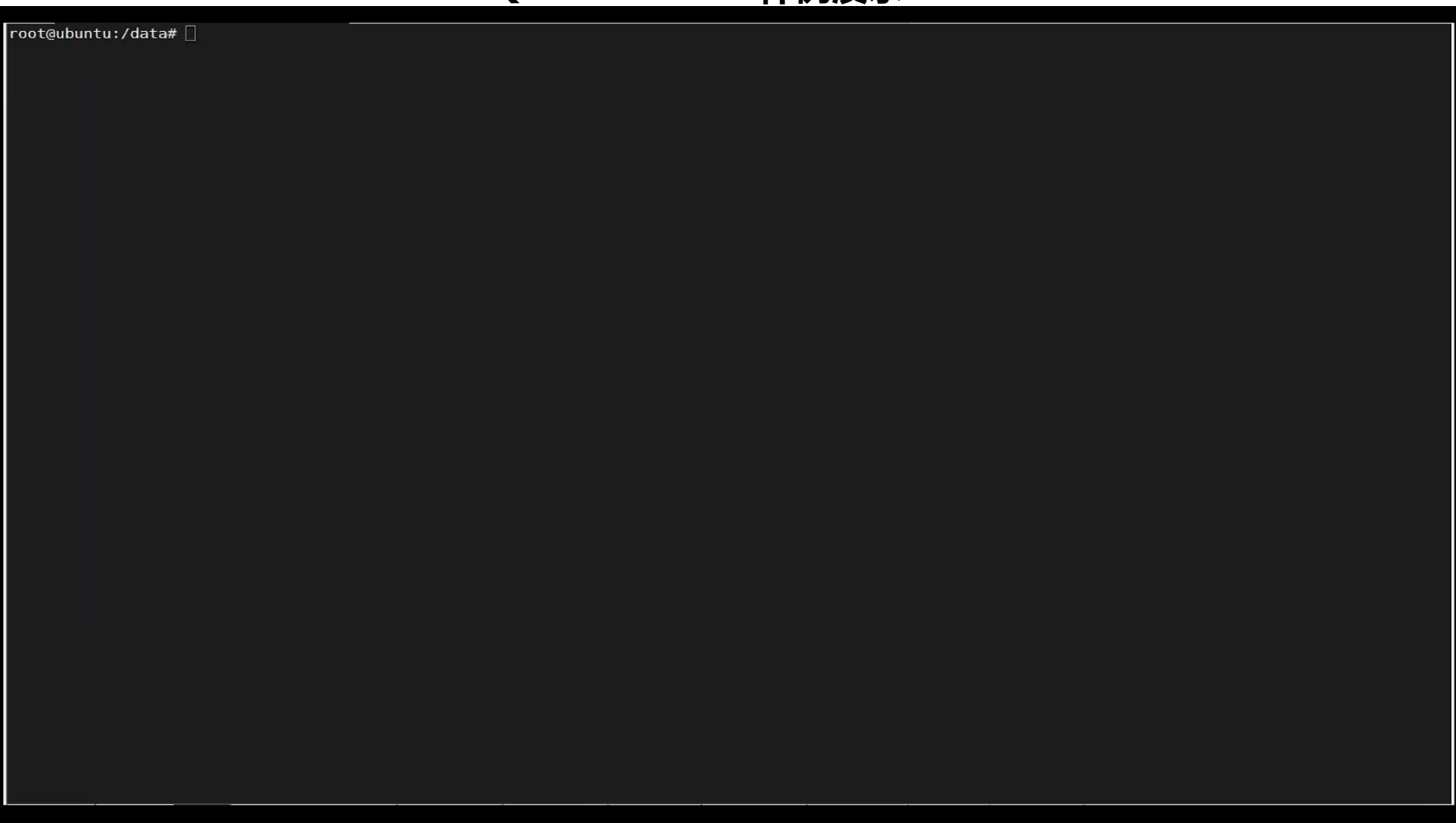
Adapt
模型适配

Evaluate
精度评估

Quantize
PTQ 优化

Deploy
部署导出

Qwen3.6-MoE 样例演示



AMCT-Agent优化 workflow

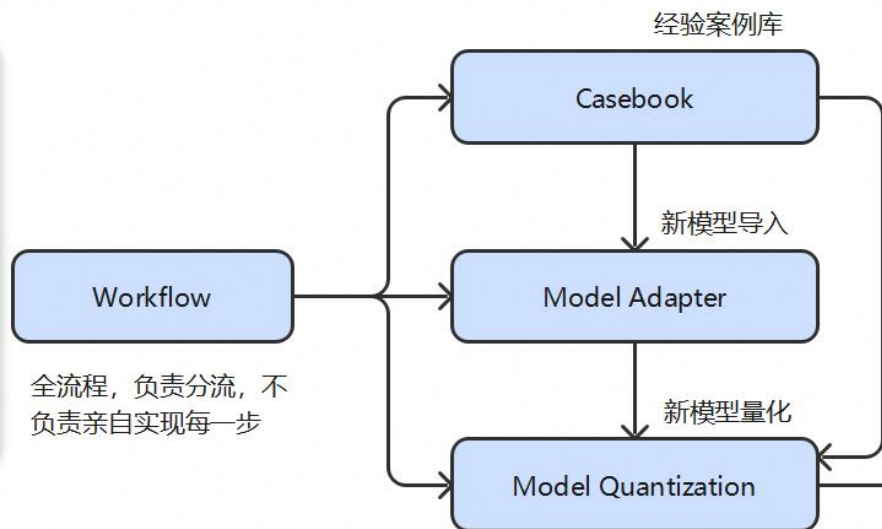
整体流程

- **查现状**: 当前工具中指定的模型适配现状确定;
- **判阶段**: 确定新模型导入或者量化的子Agent选择;
- **串流程**: 借助Quant Tools进行量化实验;
- **出交付**: 导出量化权重和量化方案介绍文件;
- **沉经验**: 更新casebook, 形成后续复用资产;

模块介绍

- **Model adapter**(只做新模型接入)
 - 模型注册
 - BF16 基线测试与关闭量化后的浮点等价测试
 - 最小PTQ冒烟
- **Model quantization**(量化阶段总控)
 - **量化方案推荐**: 根据Casebook给出量化方案, 或用户指定
 - **直转量化评测**: 基于已有量化方案做直转量化测试
 - **量化算法推荐及验证**: 根据Casebook推荐量化算法, 并验证
 - **量化权重导出**: 导出量化部署权重及方案说明文档

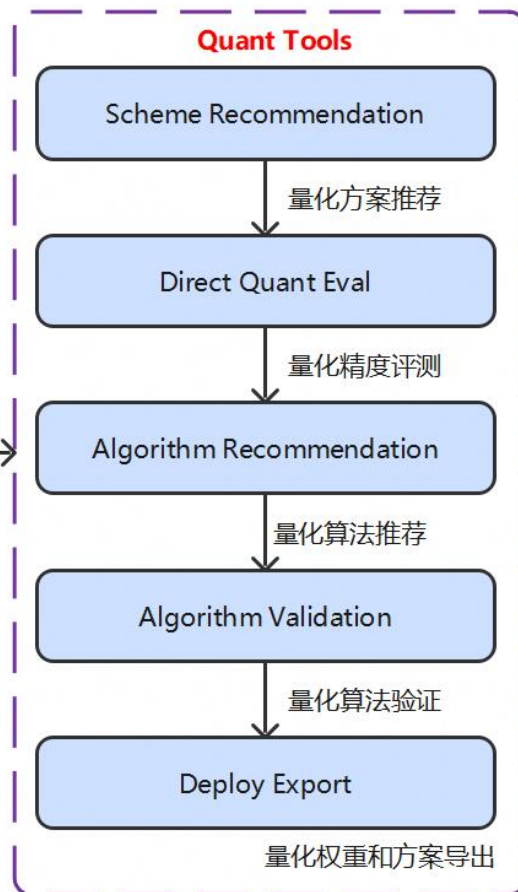
<https://gitcode.com/cann>



流程优势

- **易用性**
 - 流程简单清晰
 - 集合适配、量化、算法优化、导出, 整体性强
- **节省上下文**
 - **Repo-Map**: 新模型/新算法导入无需全局扫描所有函数, 对关键锚点文件扫描, 节省上下文, 同时了解工具整体框架

项目	内容
最近校验日期	2026-04-17
使用方式	先读 repo-map -> 抽查锚点 -> 只更新受影响 section
适用场景	新模型适配与 PTQ 流程定位

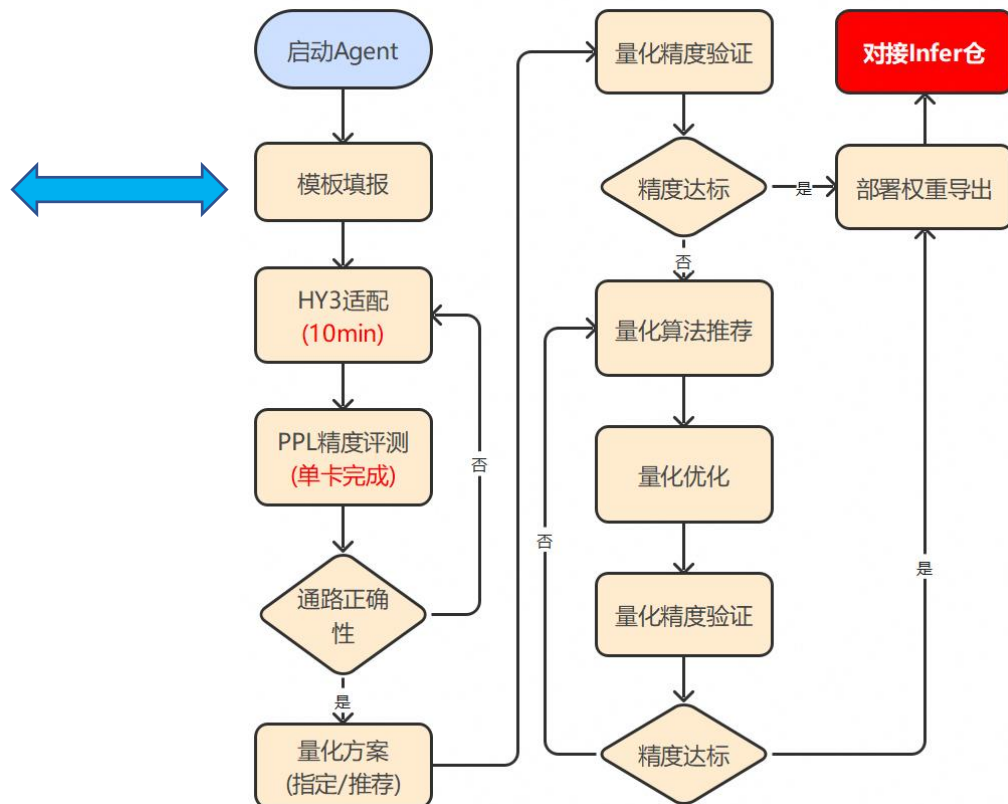


AMCT-Agent 实践样例

面向950PR/DT的Hunyuan3端到端模型量化部署 workflow

通过“模板填报 + Agent 编排”的方式完成标准化闭环，用户只需补充关键信息，Agent 即可按模型状态自动推进下一步工作

- [必填] 模型权重路径: 指定HY3权重路径
- [必填] 期望的 `model\_name`: 自定义: 例如 “hy3”
- [必填] 远程环境: 指定已有环境或者agent自行创建
- [可选] 模型结构: dense / MoE / 其他
- [可选] 特殊结构: 无 / MLA / linear attention / Mamba / 其他
- [可选] 当前阶段: 未知 / 未适配 / 已适配未量化 / 已有量化结果 / 准备 deploy
- [可选] 目标: 先打通 BF16 / 完成 model-adapter / 完成第一轮量化 / 一直到 deploy
- [可选] 是否允许复用已有结果: 是 / 否
- [可选] 量化偏好:
 - `quant\_target`: 指定 moe(A8W4); attn-linear(A8W8)
 - `quant\_dtype`: 指定 mxfp
 - `bits`:
 - `algos`:
- [可选] deploy 期望:
 - 是否要求最终导出权重:
 - 是否需要 infer 仓说明文档:



➤ 量化格式差异

950PR/DT系列芯片基于新增支持业界标准FP8/MXFP8/MXFP4等低数值精度数据格式MX (纯cube), fp8 (CV mix), MX更方便做多流并行, 计算和搬运的开销都会更小, 在计算bound的场景, 对比FP8收益明显

<https://gitcode.com/cann>

➤ FP8/MXFP8 W8A8 + MoE W4A8推理

基于950PR/DT平台, 支持了原生FP8量化, 同时也支持了使用MXFP8替换原生FP8格式的Linear模块, MOE部分替换MXFP4, 在Prefill和Decode高吞吐场景可以进一步提升算力利用率, 提高整体性能。

用户只需填报模板, Agent自动进行适配

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



AMCT 有奖社区任务



有奖互动答题



加入AMCT工具群同步进展



上CANN社区获取干货



关注CANN公众号获取资讯